Hello everyone and welcome to our course on building an **escape room game** with Python!

In this course, we're going to learn about **object-oriented programming** and use those skills to build an escape room game. This will give us a chance to learn some new skills, strengthen our old skills, and learn how to build a fully functional game from scratch.

The best part is that we can do this all using **native Python** code; there's no need to learn how to use any fancy libraries or GUIs and the code we write will be simple enough that even a basic knowledge of Python will suffice to get you started!

We chose this topic because object-oriented techniques are used in **many programming languages**, some of the more prominent being Python, Java, Swift, and C++, although there are many more. Once we understand how **classes** and **objects** work, we can apply that knowledge to all object-oriented languages, giving us a powerful set of tools and opening up many new opportunities.

We will solidify the concepts learned here by building a simple game together. **Games** are often implemented using object-oriented programming and are fun to build and use afterward, so this is a great way to learn an important concept and add to our **portfolios** at the same time. So without further ado, let's get started!

© 2022 Zenva Pty Ltd. All rights reserved.

Object-oriented programming is a style of writing code in which we use **objects** to **store** and **manipulate data**. If you have any programming experience, you probably know how to store data in variables and use functions to run blocks of code and retrieve/manipulate that data.

Object-oriented programming still uses variables and functions to store and run code; the difference is that we **encapsulate** all of the **code within objects and classes** and use these to group together related variables and functions.

Let’s dive deeper into some technical definitions:

- **Objects**: data structures that have **state** and **behavior**.
- **Object state**: a collection of related variables (called **fields**) that represent the **properties** of an object.
- **Object behavior**: is a suite of functions (called **methods**) that are typically used to retrieve, store, and manipulate the object’s **state**.

Objects are created through **initializers**, special functions that set up an object’s initial state (i.e., initial values for the fields). We **define** the object’s fields and methods using a **class**, a block of code that acts as an object’s blueprint. Within a class, we lay out the variables’ names and initial values as well as implement any functions that the object should be able to run.

We use the initializer to create objects that have the class’s properties and can call the class’s functions. We can create **many instances** of a class (objects), all of which must have the same set of fields and methods but can hold **different values**.

If we think about a class as being a **recipe** of, say, a cake, an object is the actual cake that is baked from that recipe as the recipe specifies how to bake a cake and we can use that to bake many cakes. For our project, we will use several objects to represent **components** of a game and we will outline what they should look like and how they work using classes.

© 2022 Zenva Pty Ltd. All rights reserved.

We want to create an escape room game that will make use of Python's object-oriented capabilities. The basic premise of escape room games is that we are **trapped** in a room filled with various objects and need to solve some **puzzle** to escape.

Interacting with objects often provides us with **clues** about how to escape and it's up to us to piece together all the information in order to escape the room. In our case, we'll keep it simple; we escape the room by entering a **3-digit code** to unlock the door but if we guess incorrectly 3 times, the door locks forever and we lose the game.

Our room will contain **5 items**, each of which can be looked at, smelled, and touched, for a total of **15 interactions**. Each interaction will tell us something about the item; some will provide information about the lock code, and some will just describe the item to distract the player.

By **piecing together** all of the clues, the player will have enough information to guess the 3 digits and their order to enter the code, thus escaping and **winning** the game.

© 2022 Zenva Pty Ltd. All rights reserved.

To avoid dealing with any GUI libraries, we will make this game a **text-based** game. It will run entirely within the **Terminal/CMD Prompt** and will work by printing text prompts to the console and capturing user text responses.

At each step, the program will print a **list** of **actions** the player can take and the player must select an action. The program will then process the user's selection, decide what the next set of actions are, and repeat the process until the user either guesses the code correctly, fails to guess the code 3 times, or prematurely terminates the game.

We will **implement** 3 distinct **classes** to represent the components of our game, as follows:

Class	**Component**
GameObject	Represents an item that the player can interact with
Room	Represents the room from which we're trying to escape; contains the items
Game	Represents an instance of our game that contains our room

We will keep things simple but in a more sophisticated version of our game, the Game class could contain multiple rooms, multiple levels, and even multiple players! Each of these classes will have distinct fields and methods and will be instantiated at least once. We will get into the details of each class and the representative objects in the Project Implementation section of the course.

© 2022 Zenva Pty Ltd. All rights reserved.

In this section of the course, we build our game from start to finish. We will start by learning how to print text to the console and capture user **text input**. Then, we will implement some custom **classes** using our newly learned object-oriented techniques and following the specifications outlined in our UML diagrams.

Once we have written the basics of our classes, we can implement the class **methods** that dictate the game functionality based on Flowcharts. Finally, we will write some **unit tests** and do some integration testing to ensure the program is functioning properly.

Text Input and Output

Before we begin implementing our game, we need to know how to produce text output and receive user input as our entire game revolves around these functions. We can do so through the use of the **print()** and **input()** functions, respectively.

Let's start a **new project** to house all of our game code. Feel free to **use any Python environment you desire**, as long as it has a text editor to write code and a Terminal/CMD Prompt environment to run the code.

For this course, we're using **Replit** as it provides an environment that runs in the **browser** and requires no prior installation or setup. You can simply create a free account, create a new project, and start writing and running code immediately. The website can be found here: https://replit.com/

Regardless of which environment you choose, we recommend calling your project something like "EscapeRoom" and creating a **file** called "**main.py**". This will house all of our game code. Once this is done, do a test run to make sure everything is set up.

Navigate to your project folder through Terminal/CMD Prompt and enter the **run** command or in Replit, simply press the green **Run button** at the top of the screen. This will run any code in main.py (which won't do much for now as we have no code there yet); we will repeat this process any time we want to run our code in the future.

Now that our project is set up, we can learn to use the **print()** function. This is the easiest way to **output text** to the Terminal/CMD Prompt, often referred to as the **console**. We simply put whatever we want to print within the parentheses like so:

```
print("Welcome to our escape room!")
```

If we run that code, we should see the message: "Welcome to our escape room!" appear in the console. We can put whatever **variable** or **literal value** we want in a print() function and it will either print the value or, if we put an object into the print statement, it will print a brief description and its memory address.

Go ahead and try to print a different string, a number, or a True/False value and see what output you get. The print() function is very flexible; one way that we can print **multiple items** is to **separate** them with **commas** like so:

```
print("Welcome to our escape room!", "Guess the 3-digit combination to escape!")
```

All items within the parentheses are printed next to each other but each new call to print() puts the value(s) on a new line.

© 2022 Zenva Pty Ltd. All rights reserved.

To handle **user text** input, we can use the **input()** function. This works by **displaying** a **prompt**, pausing the program's execution until the user enters some text in the console and presses enter, then returning said text. For example, if we wanted to get the user to enter their name, we could do something like this:

```
name = input("What is your name?")
```

If we run that code, the program will output the text: "What is your name?", **wait** for the user to type out their name and press enter, and then **store** the text in the variable "name". The resulting text is always stored as a **string**. Note that there are no spaces or new lines between the prompt and the user's input. We can fix that by adding a space or the newline character, "**\n**":

```
name = input("What is your name?\n")
```

This is a **special character** that doesn't appear in the actual string but forces the caret onto a **new line**. We'll make use of this technique alone with input() functions a lot in our program. Try to prompt the user to answer another question by using another call to input() and when you're ready to move on, we can begin writing the actual project code!

© 2022 Zenva Pty Ltd. All rights reserved.

Now that we understand how to handle text input and produce text output, let's start writing some actual project code.

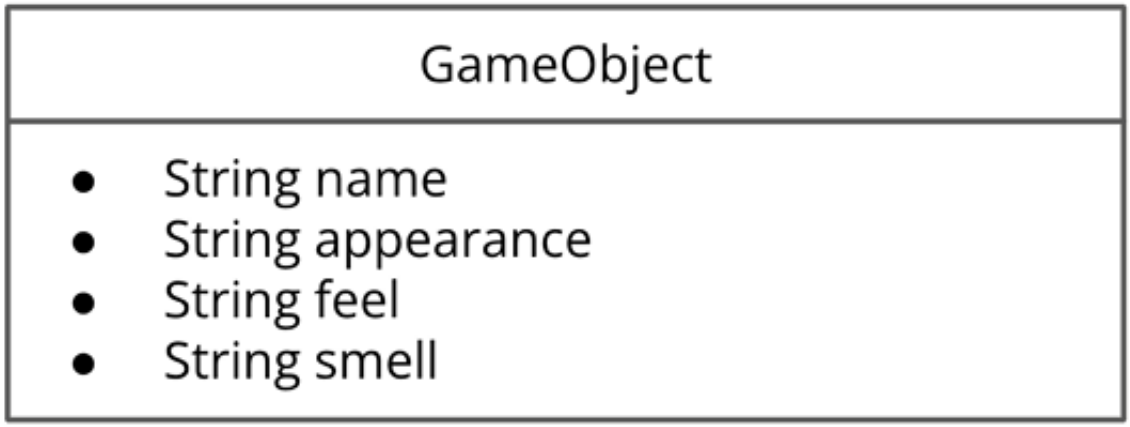
We know that we're going to use 3 custom objects (GameObject, Room, and Game) and each of them will need its own class. Let's start with the **GameObject** as we'll need this for the other two classes.

When implementing a class, we start with the **"class"** keyword followed by the class name and a colon:

```
class GameObject:
```

The class name is typically in **camel case** rather than having the words separated with an underscore. From here, we can add any fields that the class might have. Remember, **fields** are kind of like **properties** of the class, so which properties will a GameObject have? We will use 4 properties: **name**, **appearance**, **feel**, and **smell**, all of which will be strings.

We can outline how a class is going to look using a **UML** (Unified Modeling Language) **diagram**. With just the fields, our GameObject UML diagram will look like this:



This shows the **names** of the fields and the **type** that they're expected to hold. We can set these fields up in the initializer or set them with some default values like so:

```
class GameObject:
    name = ""
    appearance = ""
    feel = ""
    smell = ""
```

These are just **default** values, however. We don't necessarily want to set up every instance of GameObject with these empty strings; sometimes, we will want to set custom values. We can do so using an **initializer**, a special function used to set up the initial values for the fields of a class. It has special syntax and looks like this:

```
class GameObject:
    name = ""
```

© 2022 Zenva Pty Ltd. All rights reserved.

```
    appearance = ""
    feel = ""
    smell = ""

    def __init__(self, name, appearance, feel, smell):
        self.name = name
        self.appearance = appearance
        self.feel = feel
        self.smell = smell
```

Besides the underscores and the "**self**" parameter, it acts very similarly to a regular function in that we can add parameters and set values in a function body. The "self" parameter is added to **every** method in a class (there are exceptions but they're beyond the scope of this course). **This allows us to access any of the fields or methods in the class**; as you can see, we're using it to set the 4 fields and distinguish them from the parameters of the same name.

As we are setting values for each of the parameters in the initializer, we don't have to include them above so this is equally as valid and has the same result:

```
class GameObject:

    def __init__(self, name, appearance, feel, smell):
        self.name = name
        self.appearance = appearance
        self.feel = feel
        self.smell = smell
```

We can use the initializer to **create** new **instances** of the class in the form of **objects**. If we wanted to create a new GameObject, we could do so like this:

```
game_object = GameObject("Chair", "Some appearance", "Some feel", "Some smell")
```

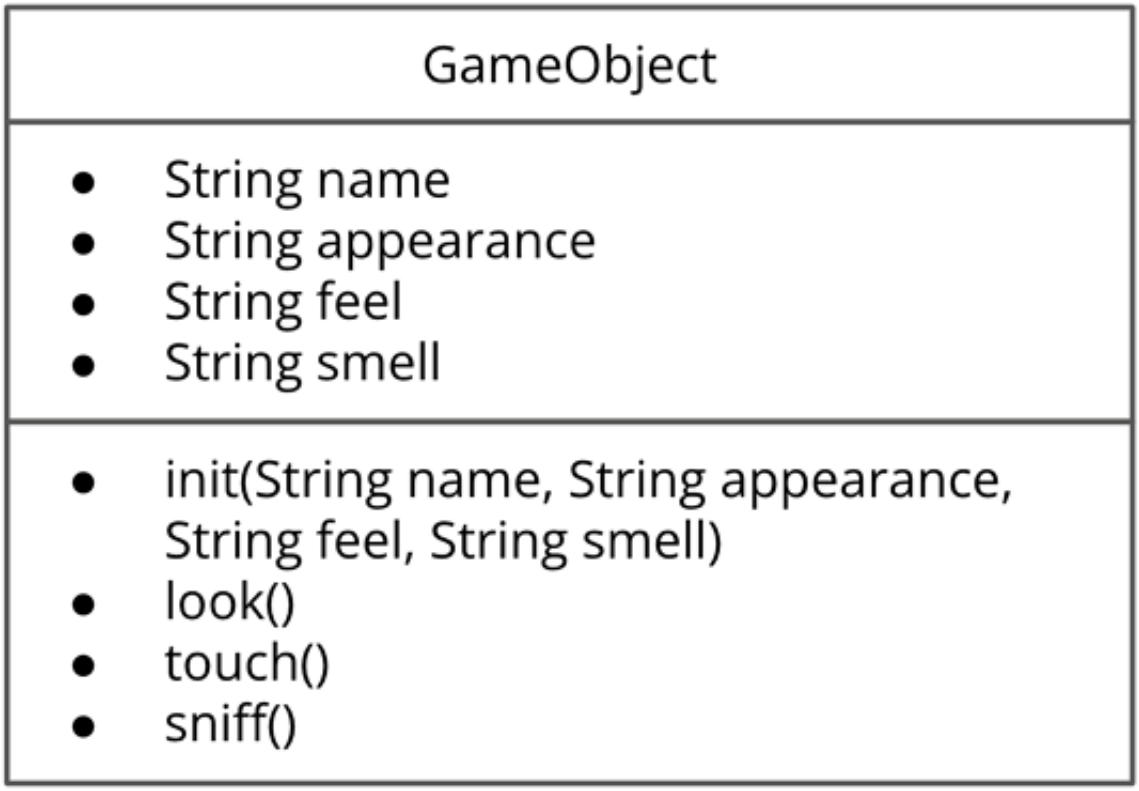
Note that when using the initializer, we don't include the underscores or add anything for the self parameter. This is just the syntax that we use. Our **game_object** is now a variable of **type GameObject** with name, appearance, feel, and smell set by the initializer. If we want to access or modify these fields, we can do so with a dot syntax:

```
game_object.name = "Knife"
print(game_object.name) # Outputs "Knife"
```

© 2022 Zenva Pty Ltd. All rights reserved.

Next up, we want to add some methods to our class. These are **functions** that belong to a class and give the **object** some **behavior** that it can execute. For GameObject, we want 3 methods: **look()**, **touch()**, and **sniff()** which will each return a string with the appearance, feel, and smell, respectively.

Adding them to our **UML diagram** yields this:



This shows the method names, any parameters with types that they need, as well as any return values expected from them. Implementing them in **GameObject** will look like this:

```
class GameObject:
    def __init__(self, name, appearance, feel, smell):
        self.name = name
        self.appearance = appearance
        self.feel = feel
        self.smell = smell

    def look(self):
        return f"You look at the {self.name}. {self.appearance}\n"

    def touch(self):
        return f"You touch the {self.name}. {self.feel}\n"

    def sniff(self):
        return f"You sniff the {self.name}. {self.smell}\n"
```

Note how each of the methods has “**self**” as a **parameter**. Again, we will need to add this as a parameter for each method we implement in a class but don’t need to pass anything into it when

© 2022 Zenva Pty Ltd. All rights reserved.

calling it. It gives **access** to **any fields or methods** belonging to that **class**.

Also, see how each of the strings is preceded by an "**f**" and contains {self.name} and then {self.field}. This provides a way to **input** variables into a string so that they are evaluated and **converted** to a string at that position within the parent string. For example, calling the look() method will result in this string being returned:

```
"You look at the Chair. Some appearance"
```

Just like with variables, we can **call methods** of an object using **dot syntax**:

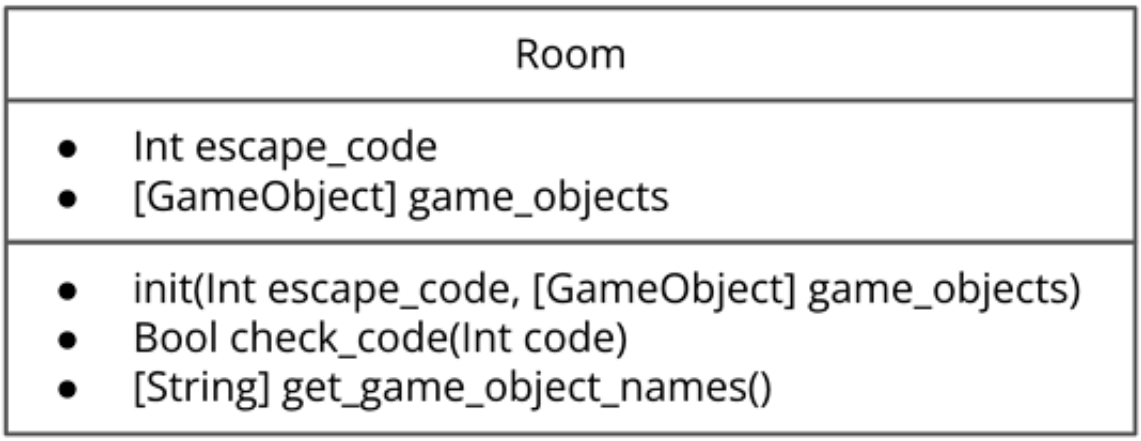
```
game_object.sniff()
```

That's it for our GameObject class! Try creating some other GameObjects and playing around with the fields and methods.

© 2022 Zenva Pty Ltd. All rights reserved.

Now let's move on to our next class, the **Room**. This will need an **escape code** and a **list** of **GameObjects** as fields. It will also need an initializer to set these fields up along with a **method** to check to see if a code **matches** the Room's code along with a method to **fetch** a list of the **names** of the GameObjects.

We can specify all of that in this **UML** diagram:



Take note of the **parameters** (with types) and the **return types** of any methods specified. When adding the **fields**, we can **add** them and **set** them up in the **initializer** like this:

```
class Room:
    def __init__(self, escape_code, game_objects):
        self.escape_code = escape_code
        self.game_objects = game_objects
```

Or first, create them with **default values** then set them in the initializer like this:

```
class Room:
    escape_code = 0
    game_objects = []

    def __init__(self, escape_code, game_objects):
        self.escape_code = escape_code
        self.game_objects = game_objects
```

Either way, the results are the same. Our **check_code** method simply returns **True** or **False** based on whether the codes match:

```
def check_code(self, code):
    return self.escape_code == code
```

© 2022 Zenva Pty Ltd. All rights reserved.

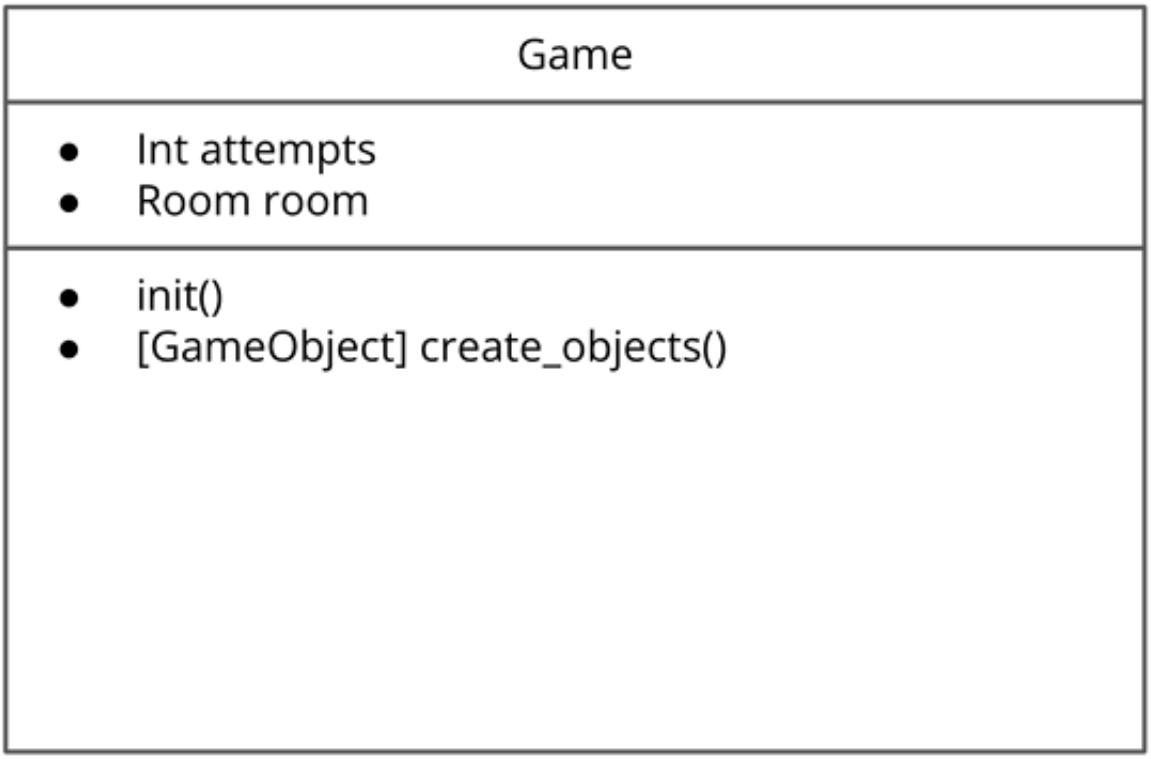
Whereas the **get_game_object_names loops** through the list of GameObjects, extracting the names to then return them as a list:

```
def get_game_object_names(self):
    names = []
    for object in self.game_objects:
        names.append(object.name)
    return names
```

That’s all we need to do for the Room class! Let’s move on to the final class.

© 2022 Zenva Pty Ltd. All rights reserved.

This is going to be more complex than the previous two because this will contain most of the game logic. We'll only need a couple of fields: an int representing the **number** of **attempts** the user has taken at guessing the code and a room with the **escape code** and **GameObjects**. We'll also need an **initializer** to set these up along with a method called **create_objects**() to create the GameObjects to pass into the Room:



This is just the top half of the UML diagram; there are several other methods in Game that we will implement a little later. First, we can **create** our **Game class** and set up the attempts like so:

```
class Game:
    def __init__(self):
        self.attempts = 0
```

Next, we will create a list of GameObjects and return them via the **create_objects** method. It does require some hard thinking to come up with which **GameObjects** to create and how to word them in such a way as to provide enough **clues** for the user to be able to guess each digit of the 3-digit escape code correctly. Feel free to use your own set of GameObjects if you want to build your own puzzle:

```
def create_objects(self):
    return [
      GameObject(
        "Sweater",
        "It's a blue sweater that had the number 12 switched on it.",
        "Someone has unstitched the second number, leaving only the 1.",
        "The sweater smells of laundry detergent."),
      GameObject(
        "Chair",
```

© 2022 Zenva Pty Ltd. All rights reserved.

```
        "It's a wooden chair with only 3 legs.",
        "Someone had deliberately snapped off one of the legs.",
        "It smells like old wood."),
      GameObject(
        "Journal",
        "The final entry states that time should be hours then minutes then seconds (
H-M-S).",
        "The cover is worn and several pages are missing.",
        "It smells like musty leather."),
      GameObject(
        "Bowl of soup",
        "It appears to be tomato soup.",
        "It has cooled down to room temperature.",
        "You detect 7 different herbs and spices."),
      GameObject(
        "Clock",
        "The hour hand is pointing towards the soup, the minute hand towards the chai
r, and the second hand towards the sweater.",
        "The battery compartment is open and empty.",
        "It smells of plastic."),
    ]
```

Can you guess the escape code based on these clues? Try it for yourself before instantiating the Room below.

We can replace our initializer with this:

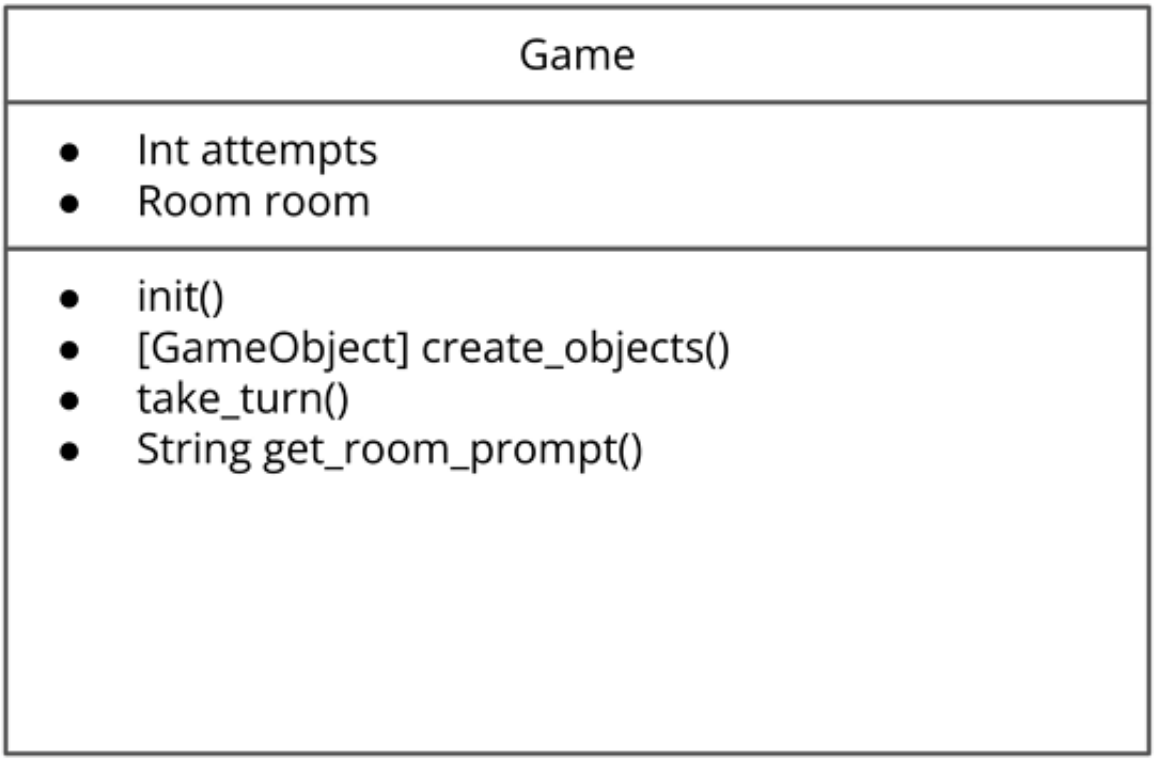
```
def __init__(self):
    self.attempts = 0
    objects = self.create_objects()
    self.room = Room(731, objects)
```

The code is **731** because the **sweater** gives us the number 1 (number 2 was removed)**,** the **chair** gives us the number 3 (one leg was removed), and the **soup** gives us number 7 (7 herbs and spices). Also, the clock points the **hour** hand at the soup (**7**), the **minute** hand at the chair (**3**), and the **second** hand at the sweater (**1**) while the journal tells us the time should go **H-M-S** so 7-3-1, hence the final code.

© 2022 Zenva Pty Ltd. All rights reserved.

Next up, we want to start implementing take_turn(). This method acts as a **game loop** with the bulk of the logic flowing through it. The first step in this method is to get the initial room prompt telling the user to either enter a 3-digit code or choose from one of the items listed below.

Adding these to our **UML** diagram yields this:



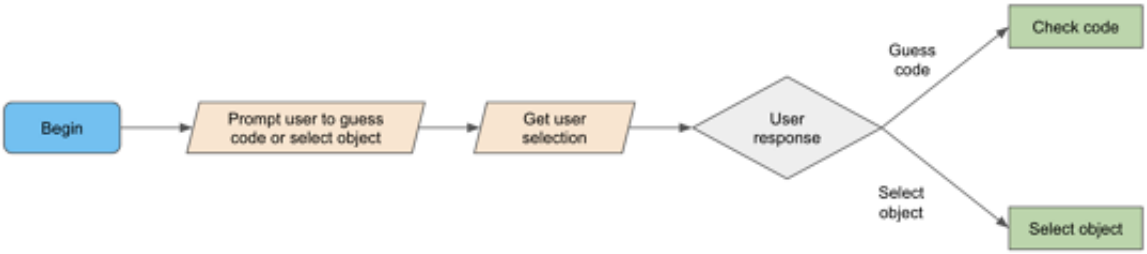
This is our implementation of **get_room_prompt**:

```
def get_room_prompt(self):
    prompt = "Enter the 3-digit lock code or choose an item to interact with:\n"
    names = self.room.get_game_object_names()
    index = 1

    for name in names:
        prompt += f"{index}. {name}\n"
        index += 1
    return prompt
```

It tells the user to enter the escape code or select an item and **prints** each of the item names beside a **number** (1-5) to allow for easy user selection. Now that we have the prompt, we can start implementing take_turn according to this logic:

© 2022 Zenva Pty Ltd. All rights reserved.



This first displays the **prompt** then gets the **user selection**, and either executes the guess code logic or the select object logic based on the user's choice. Implementing the first step looks like this:

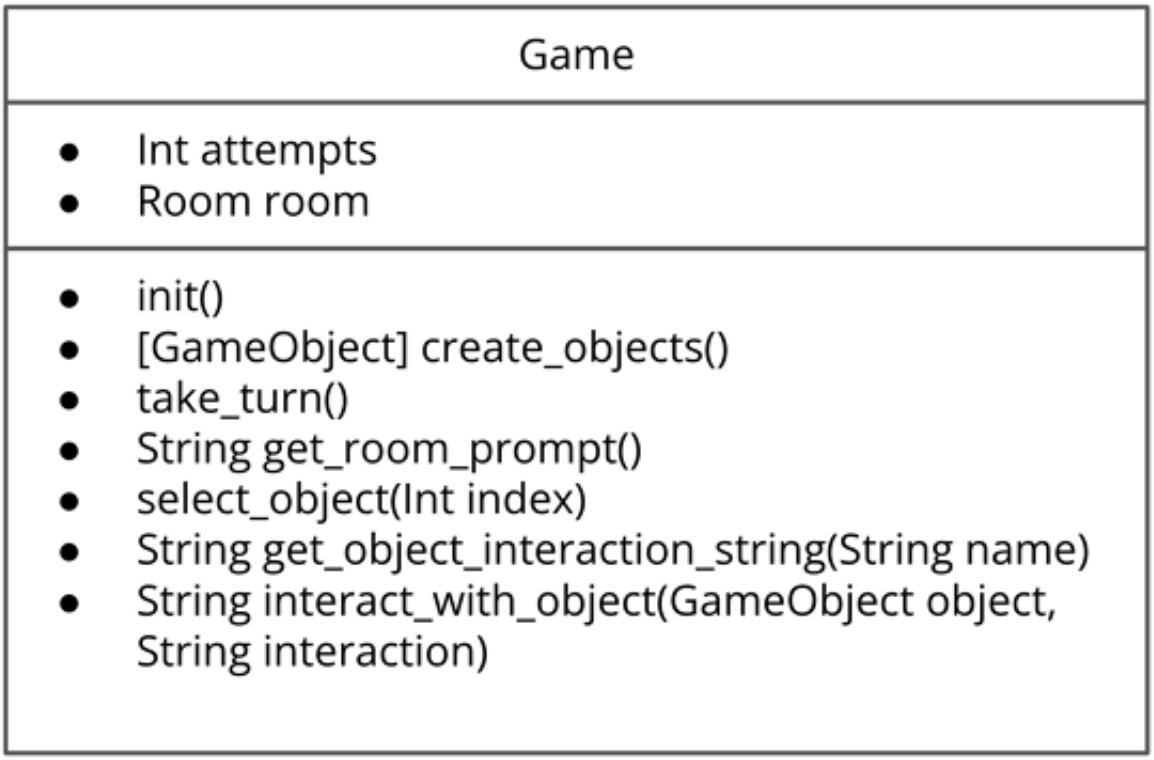
```
def take_turn(self):
    prompt = self.get_room_prompt()
    selection = input(prompt)
```

We can **run** this method to at least see the prompt by creating a Game instance and calling take_turn:

```
game = Game()
game.take_turn()
```

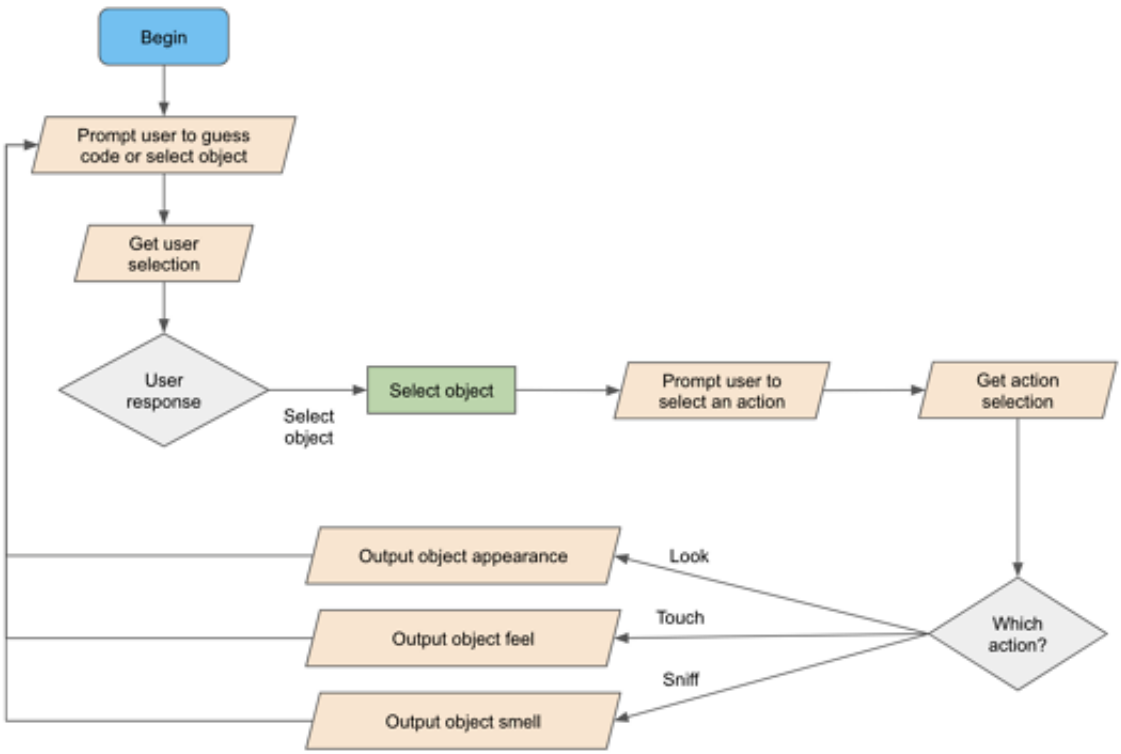
© 2022 Zenva Pty Ltd. All rights reserved.

Now we want to implement the ability to interact with a single GameObject so we need to be able to select it. To do that, we will implement a select_object() method and a couple of helper methods: **get_object_interaction**() and **interact_with_object**() according to this **UML** diagram:



The idea here is that once an object is selected, we present a **list** of **possible interactions** to the user (look at, touch, or sniff), have them choose an interaction, and then present them with the pertinent information (**appearance**, **feel**, or **smell**). This process is outlined in this **flowchart** (ignoring the guess_code logic for now):

© 2022 Zenva Pty Ltd. All rights reserved.



Let's start with the easy part: constructing a **message** to prompt the user to **select** an **action**. We can do so by implementing **get_object_interaction_string**():

```
def get_object_interaction_string(self, name):
    return f"How do you want to interact with the {name}?\n1. Look\n2. Touch\n3. Smel
l\n"
```

We're inserting the GameObject name into the string to give it a more custom feel but otherwise, we are telling the user to select **1, 2, or 3** to interact with the object, putting the options on different lines. To see this in action, we can call this method from select_object and then call select_object from take_turn:

```
def select_object(self, index):
    selected_object = self.room.game_objects[index]
    prompt = self.get_object_interaction_string(selected_object.name)
    interaction = input(prompt)
```

Here, we **select** an object at the specified index, get the **prompt** for that object's name, and present it to the user, **storing** their choice in the variable **interaction**. You can print it out if you want to verify that the selection is taking place. We can then call select_object from take_turn:

```
def take_turn(self):
    prompt = self.get_room_prompt()
    selection = int(input(prompt))
    self.select_object(selection - 1)
```

© 2022 Zenva Pty Ltd. All rights reserved.

For now, we are making a couple of assumptions:

1. The input from the user is a number that can be converted to an integer
2. The selection integer is >= 1 and <= 5

Note how we have to **decrease** the selection by 1 because we want a number **0-4** for **indexing** instead of 1-5. Running the game will now prompt the user for a GameObject selection and then prompt the user to select an action.

© 2022 Zenva Pty Ltd. All rights reserved.

The next step is to actually **interact** with the **GameObject** to get the correct information string and print it out. We can do this by implementing interact_with_object:

```
def interact_with_object(self, object, interaction):
    if interaction == "1":
        return object.look()
    elif interaction == "2":
        return object.touch()
    else:
        return object.sniff()
```

This takes in a GameObject and the interaction (either 1, 2, or 3 depending on whether the user wants to look at, touch, or sniff the object). There's no need to convert this input to an integer as we're not using it for list indexing.

Additionally, let's just assume that if the user enters anything **other** than 1 or 2, we are going to pretend it's a **3**. This lets the GameObject's methods do the heavy lifting in terms of constructing a string so now we can fetch the string and **print** it out in select_object for this complete implementation:

```
def select_object(self, index):
    selected_object = self.room.game_objects[index]
    prompt = self.get_object_interaction_string(selected_object.name)
    interaction = input(prompt)
    clue = self.interact_with_object(selected_object, interaction)
    print(clue)
```

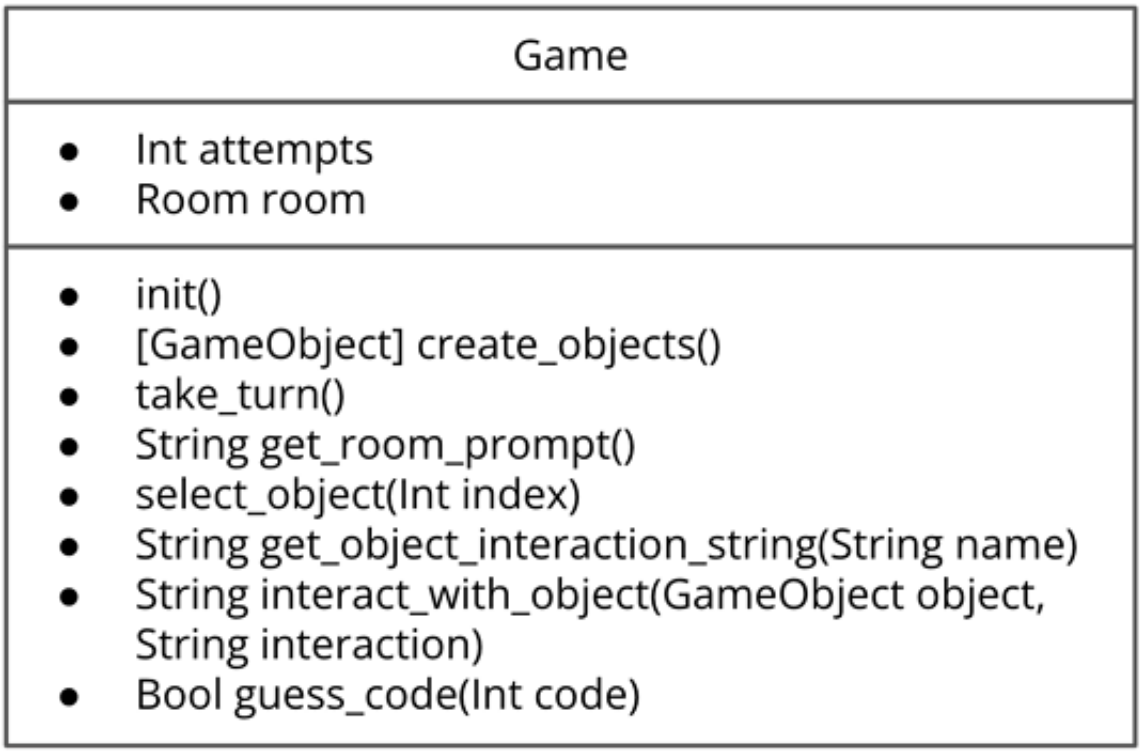
Running this will now print out the correct information based on the user's selections. However, the game stops running as soon as the user interacts with an object; we want to **keep** it **running** until the user **wins**, **loses**, or **quits** the game. We can do this and add some input-verification logic in **take_turn**:

```
def take_turn(self):
    prompt = self.get_room_prompt()
    selection = int(input(prompt))
    if selection >= 1 and selection <= 5:
        self.select_object(selection - 1)
        self.take_turn()
```

This ensures that the object selection will only take place if the input is a number between **1-5**. It also ensures that the game will take another turn right after an object interaction finishes, keeping the game going.

© 2022 Zenva Pty Ltd. All rights reserved.

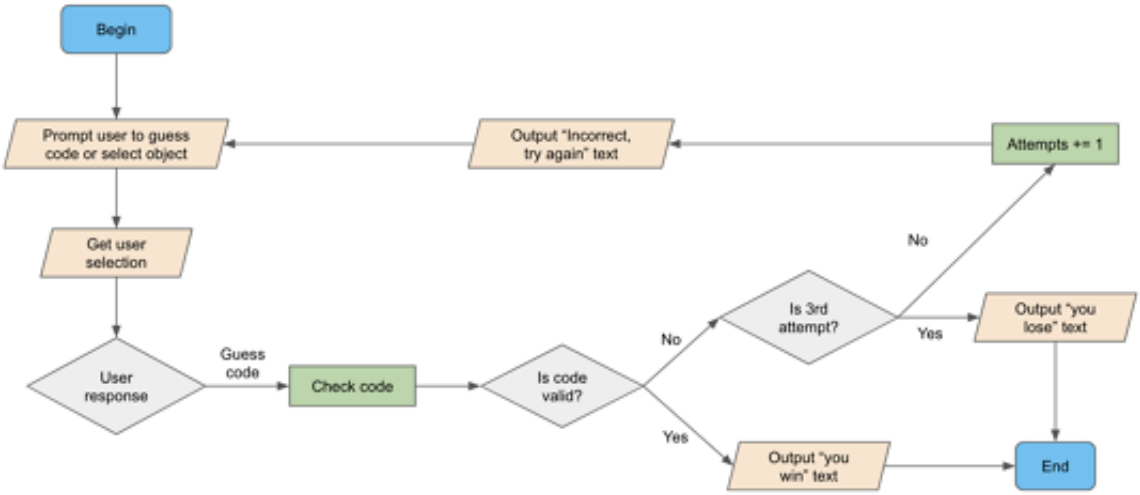
Now we can implement the final logic: guessing a code. This is a fairly simple process as we already have a method to verify if a code matches a Room's code but we still need a method to **call** this and execute some more logic outlined here in the complete **UML** diagram:



The logic here is simple enough:

1. Check if the **codes match**
2. If they do, we **win** the game and exit
3. If **not**, we **increase attempts** by **1**
4. If **attempts >= 3**, we **lose** and exit
5. If not, we let the user know the codes didn't match and take **another turn**

This process is illustrated in this **flowchart**:



© 2022 Zenva Pty Ltd. All rights reserved.

To start, we can execute the checking and attempts-increase logic in **guess_code**:

```
def guess_code(self, code):
    if self.room.check_code(code):
        return True
    else:
        self.attempts += 1
        return False
```

Now we can add the extra case-checking in **take_turn**():

```
def take_turn(self):
    prompt = self.get_room_prompt()
    selection = int(input(prompt))
    if selection >= 1 and selection <= 5:
        self.select_object(selection - 1)
        self.take_turn()
    else:
        is_code_correct = self.guess_code(selection)
        if is_code_correct:
            print("Congratulations, you win!")
        else:
            if self.attempts == 3:
                print("Game over, you ran out of guesses. Better luck next time!")
            else:
                print(f"Incorrect, you have used {self.attempts}/3 attempts.\n")
                self.take_turn()
```

This ensures that if the user hasn't entered a number from **1-5**, they're guessing the code. If it **matches**, they **win** the game, otherwise, we check to see if they have run out of attempts. If they have, they lose and we exit, otherwise, we let them know that they were incorrect and take another turn.

At this point, the **project implementation** is **complete** and should be working as expected! Run the game and try to solve the puzzle using the information contained within the object interactions. To ensure that everything is working exactly as it should, let's add some tests.

© 2022 Zenva Pty Ltd. All rights reserved.

No program is ever complete without undergoing tests to evaluate performance. Although tests can come in many forms, we will focus on two types: **unit tests** and **integration tests**.

Unit Tests

Unit tests test the **performance** of **individual** functions or **small blocks of code**. These are used to model all possible input-output scenarios and ensure that each individual part of the code will hold up under expected and unexpected conditions.

When writing the tests for an individual function or block of code, we want to consider all possible **input-output** combinations to make sure that the code is robust and has no points of failure.

Unit tests provide a **quantitative** measure of how well a program is performing.

Integration Tests

Integration tests test **how functions work together** when performing a task or set of functionality. They are used to **target procedures** or overarching functionality and consider the results of a series of functions running together rather than looking at each individual function.

Integration tests provide a more **qualitative** measure of how well a program is performing.

Why Do We Need Testing?

Ideally, we would never have to bother with tests and would write perfect code every time. However, the real world is far from ideal and inevitably, we or someone on our team will make mistakes.

Writing appropriate test coverage helps to eliminate or at least **minimize** the amount of and impact of those mistakes. Testing ensures that the code **performs properly** under all conditions and environments.

We can also use tests to ensure that any changes made to existing code **do not** break the old code. Although it can be an annoying and tedious process, testing should always be a part of your repertoire and you should never release code without testing it first.

© 2022 Zenva Pty Ltd. All rights reserved.

Not all of our code can be unit tested properly; as a general rule, anything that involves direct user interaction is a bad candidate for unit testing. This **eliminates** any of the methods that involve **print**() or **input**() function calls as these are very difficult to unit test. Instead, let's focus on testing our Room class as there are a couple of methods that can easily be unit tested and you can add any other unit tests for the other classes afterward.

We don't have a proper testing framework on hand so let's write our own tests from scratch. For us, this will involve **testing** a **value** and printing whether or not it matches an expected value. If the result is True then the test passed; if not, the test failed and we either need to rewrite the test or the code we're testing.

Let's start by creating a **class** to hold our **Room tests** and setting up a couple of rooms:

```
class RoomTests:

    def __init__(self):

        self.room_1 = Room(111, [
            GameObject(
                "Sweater",
                "It's a blue sweater that had the number 12 switched on it.",
                "Someone has unstitched the second number, leaving only the 1.",
               "The sweater smells of laundry detergent."),
            GameObject(
                "Chair",
                "It's a wooden chair with only 3 legs.",
                "Someone had deliberately snapped off one of the legs.",
                "It smells like old wood.")
        ])

        self.room_2 = Room(222, [])
```

Here, we're using a couple of the GameObject's from the game to set up room_1 and used an empty list of GameObjects for room_2.

Let's start by testing **check_code**. There are two possible scenarios for this method: either the **codes match** and we return **True** or the codes do not match and we return False. To fully encompass these possibilities, we just need **two tests**:

```
def test_check_code(self):
    print(self.room_1.check_code(111) == True)
    print(self.room_1.check_code(222) == False)
```

Technically, we aren't checking the **type** of the **parameter** so there could also be a value of None or it could be a number or a string but let's make some **assumptions** about the type of parameter being a Boolean.

Creating an instance of RoomTests and **running test_check_code** should output **two True values** meaning our tests have both **passed**.

Next, we want to test **get_game_object_names**. Again, we have two scenarios: either the

© 2022 Zenva Pty Ltd. All rights reserved.

game_objects list is **empty** and we return an empty list, or the game_objects list is populated and we return a **list** of all of the **names**. We can test both of those cases with just these two lines in test_get_game_object_names:

```
def test_get_game_object_names(self):
    print(self.room_1.get_game_object_names() == ["Sweater", "Chair"])
    print(self.room_2.get_game_object_names() == [])
```

Running this method should again print two **True** values meaning the test has passed. And that's it for our Room tests! Feel free to add more unit tests for the other classes.

© 2022 Zenva Pty Ltd. All rights reserved.

At this point, we have finished writing our game code and have written and run some unit and integration tests, so we can be sure that our project is complete and fully functional! Let's **review** our project definition and project design to make sure that we have met all criteria.

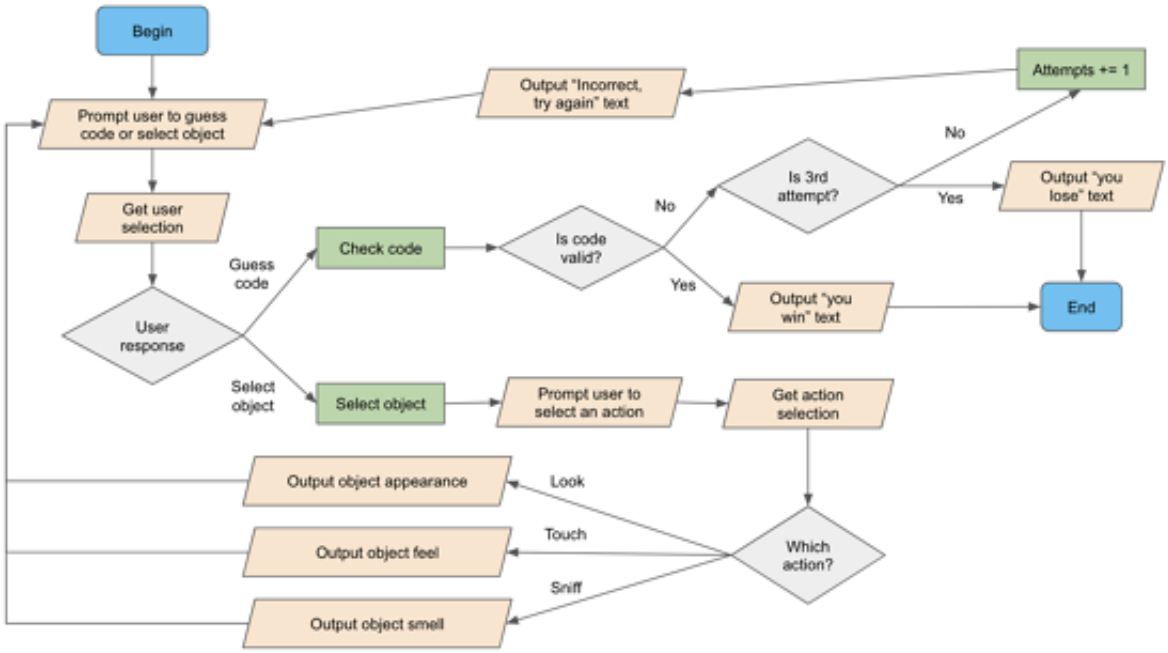
We wanted to build an escape room game that is filled with **interactive objects** and is won by correctly entering a 3-digit code. We have successfully built a game that contains at least one room filled with items, each of which can be **looked** at, **smelled**, and **touched** to provide key pieces of information.

The player can then **piece together** all of the **clues** to guess the **3-digit code** and win the game or lose if they guess incorrectly 3 times. We have fulfilled our project definition criteria here, so let's now check our project design specification.

We wanted our game to be a **text-based** game run through the **Terminal**/CMD Prompt that prints a list of possible interactions, captures the player's selection, and responds accordingly. We start by allowing the user to **select** an **object or guess** the **code**. Guessing the code executes the guess_code() function to either win or lose/get us closer to losing.

Selecting an object allows the user to then choose whether to look at, smell, or touch the object which **prints out** some information and then goes back to the initial prompt. We have designed our game according to the specifications so we have fulfilled our project design criteria as well, thus our project is complete!

As a reference, here is the **complete game flowchart**:



© 2022 Zenva Pty Ltd. All rights reserved.

In this section, we will discuss how video games in general impact individuals or society from a social standpoint, what direct or indirect effects video games have on the environment, and how economies are affected by video games. We want to treat these as thought experiments so we encourage you to take a few minutes and create a pros and cons list for each level of effect. There are no right or wrong answers here so long as you can provide a reasonable justification.

Starting with the **social impact**, we have a few pros and cons of video games from a social standpoint:

Pros	Cons
Educational	Time-consuming
Fun	Lack of physical activity
Socialize through multiplayer	Sleep disruption

Let’s take a deep dive into each one:

Educational

Many video games, such as the one we just built, require the players to solve some sort of problem in order to win. Sophisticated video games are actually not easy to play and challenge players to overcome a variety of obstacles in order to progress. Puzzle and RPG games require general **problem-solving** and working **memory skills**. FPS games require **spatial awareness** and **communication**. RTS and city-building games require **strategy** and the ability to **plan** and respond under pressure. These are just a few examples of how video games encourage players to improve certain skills.

Fun

At the end of the day, most video games are fun as people wouldn’t play them otherwise. People need some **entertainment** and downtime in their lives; it’s not reasonable to expect most people to be engaging in hyper-productive activities for every waking hour of the day.

Without some sort of a break, people often become depressed, anxious, unmotivated, and generally unwell. Video games provide temporary relief from the pressures of everyday life and **allow** players to **unwind** and **relax**.

Socialize Through Multiplayer

Video games often get a bad reputation for being antisocial devices that encourage players to set aside their social lives in order to keep playing. While this may be true for certain games, especially when played in excess, many video games have a multiplayer aspect to them and demonstrate the exact opposite.

Players can play together on a **team**, in which case **cooperation** and communication are essential to success. Even free-for-all type games are a great way for friends or even strangers to get together and play, whether virtually or in the same room. This can **strengthen** existing **friendships** or build new ones as games of any kind bring together people with common interests.

Time-Consuming

Most activities can be detrimental when done in excess; even too much exercise can lead to a variety of problems. The same is true of video games; players can spend so much time playing that they sacrifice school, careers, relationships, or other important activities such as eating, sleeping, or exercising. Of course, this can be true of any activity but video games, in particular, can be very

© 2022 Zenva Pty Ltd. All rights reserved.

addictive and hours can fly by without a player noticing.

Like most areas of life, the key is balance and being disciplined enough to not let video games interfere with what's important in life. Unless you are a professional gamer, video games are just a **recreational activity** and should be treated as such.

Lack of Physical Activity

Physical activity is important to maintaining good general health; if you neglect your body, you increase your chances of injury, disease, lack of mobility, and general physical and mental decline.

Most video games are not very physical and require little to no movement. As a result, players can end up **sitting still** for hours at a time, especially if they are engrossed in the video game and overlook the time passing by. It's generally good practice to get up and **move around** for at least a few minutes every hour and also spend some time engaged in vigorous physical activity every day.

On top of not moving much, video games typically encourage the same set of movements over and over again which can lead to **repetitive stress injuries** so creating an ergonomic gaming space is encouraged to help alleviate some of these issues.

Sleep Disruption

Video games encourage **staring** at **screens** for extended periods of time, whether it's a TV, a computer, a phone, or some other device. In general, this isn't a great practice but coupled with the fact that players often play video games close to the time they go to sleep can lead to sleep disruption. If video game playtime is not managed well, players often **stay up playing** video games later than they would otherwise, further disrupting sleep schedules.

Finally, a lack of movement associated with playing video games can cause restlessness unless players find some physical outlet for their energy elsewhere. Forgoing healthy sleep habits can lead to a host of physical and mental health issues both short and long-term.

However, players can reduce these issues by turning off the video games an hour before bed, being **disciplined** about how much time they spend playing video games, and **exercising** every day.

© 2022 Zenva Pty Ltd. All rights reserved.

Here we have a few possible pros and cons of video games from an **environmental** standpoint:

Pros	**Cons**
Not actively producing pollution	Energy-intensive
Some games teach awareness	Throw out old hardware

Let’s take a deep dive into each one:

Not Actively Producing Pollution

Many hobbies produce massive amounts of pollution over the long and even short term. Some activities directly pollute ecosystems such as flying, boating, etc. whereas others indirectly pollute or disrupt ecosystems such as golf (course maintenance), skiing (lift operation, ski hill maintenance), etc. In fact, many hobbies and activities require the participants to drive to and from venues which creates a lot of pollution.

On the other hand, playing video games is a **relatively harmless** pursuit as players don’t need to even leave the house to participate and besides the energy drain, playing video games is not directly polluting any ecosystem. The point is not to make those who enjoy other activities feel guilty, but rather to highlight the fact that playing video games is among the least directly harmful hobbies that can be enjoyed.

Some Games Teach Awareness

Some video games either directly or indirectly teach **healthy** environmental **practices**. For some video games, maintaining a thriving ecosystem is either the primary objective of the game or at least an essential part of winning the game.

Some other video games highlight the dangers of not caring for the Earth through being set in post-apocalyptic wastelands or having the characters suffer as a result of an unhealthy ecosystem. Although not every video game focuses on the environment, many video games encourage players to look after the environment in some fashion.

Energy-intensive

As video games become more sophisticated, the amount of **processing power** required to create and run them also **increases**. 50 years ago, video games only took up a few KBs or MBs of data and so the energy required to run them was minimal. However, video games now can take up several dozens of GBs of memory and the energy required to render HD graphics and run sophisticated AIs is thousands or even millions of times higher. That coupled with the fact that more and more people play some sort of video game only increases the total energy drain.

Video games are only going to become more sophisticated and require more processing power so we can expect the total energy demand to also keep increasing.

Throw Out Old Hardware

Video game consoles are constantly getting revamped, **upgraded**, and new models released, and with them, new versions of the games must be purchased to run on the new hardware. Even gaming PCs must be upgraded by **swapping** out old parts for newer, **more powerful components**. Once a replacement has been purchased that old hardware is likely not going to be used and is often thrown out.

Although we can help reduce waste by selling old consoles, games, and components, eventually,

© 2022 Zenva Pty Ltd. All rights reserved.

that hardware becomes so obsolete that no one will buy it. We can **recycle** some of the old hardware but not all of it can be recycled and a lot of “recycled” material actually ends up in a landfill instead. This can end up producing a lot of waste when consoles or PCs must be upgraded every 1-2 years to keep up with the latest technology.

© 2022 Zenva Pty Ltd. All rights reserved.

Here are a few pros and cons of video games from an **economic** standpoint:

Pros	Cons
Career potential	High initial cost
Creates jobs	Resource intensive
Growing industry	Service industries lose money

Let’s take a deep dive into each one:

Career Potential

Since their creation, video games have remained a source of **competition** in which players can pit their skills against others to prove who’s best. As more people flocked to video games, the pool of competitors also increased to the point where **legitimate tournaments** with cash prizes were organized and executed. Once video games shifted from just a hobby to a means to win money, a career in gaming was born.

Now, video games are a legitimate source of income for millions of gamers across the world with players and teams being funded through tournaments, sponsorships, streaming platforms, and donations. In many countries, top **professional gamers** are held in esteem equal to top professional athletes, with salaries to match.

Much like sports, esports is a highly competitive field with few rising to the very top but gaming is becoming a more lucrative industry, and becoming a top professional gamer is a position that many strive to achieve for the esteem, the enjoyment of the game, and also the money.

Creates Jobs

As the gaming industry grows, so does the **demand** for **game developers**. Besides being an exciting and interesting job, game developers can be highly paid as they climb the ranks and even develop and release their own games. Along with developers comes the need for developer support, admin, tech support, and QA staff, just to name a few linked professions. On top of this, a larger pool of developers also requires larger facilities, service industry support, housing, and transportation services, along with a lot of other **infrastructure support**.

As well, the demand for materials required to produce the hardware and software increases along with the demand for manufacturers. This creates jobs everywhere for people in a variety of industries and stimulates economic growth. As the gaming industry grows, so too does every industry either directly or indirectly linked to the hardware, software, or to the developers themselves.

Growing Industry

The **video game industry** has grown massively over the past 50 years and will continue to grow at an increasingly rapid pace. Over the past 20 years, the esports industry has grown from nothing to a highly competitive enterprise over many different multiplayer games, presenting **opportunities** for players and developers.

As stated before, along with the developers come all sorts of support staff and service industry jobs along with the necessary infrastructure. Video game companies boom with the release of each successful game and with them, chip makers and other hardware companies. Booming industries like this prop up many other industries and are, generally speaking, a net positive on any country’s **economy**.

© 2022 Zenva Pty Ltd. All rights reserved.

High Initial Cost

Gaming rigs and consoles can be **expensive** purchases which only get more expensive as the underlying technology improves. Along with the cost of the console or PC come the costs of gaming chairs, desks, mice, keyboards, controllers, and of course, the games themselves.

Recently, some games themselves have started to require **subscriptions** or **in-game purchases** to really enjoy. Once the setup has been established, there are few continued costs besides subscriptions and the cost of a new game once a player finishes their current one.

However, the barrier to entry is still quite high and there are many cheaper hobbies out there. That said, buying second-hand significantly lowers the cost and in the PC sphere, the computer can be used for work as well as gaming so could technically be considered more of a work expense.

Resource Intensive

Cutting-edge technology requires very specific materials that are often procured at high **human cost**. Metals have to be mined to make chips, displays, switches, and other pieces of hardware. When these materials are in short supply, the price skyrockets and other industries that rely on these resources can suffer.

This problem is exacerbated by the fact that many companies release new consoles every 1-2 years and PC gamers are constantly upgrading their gaming rigs by swapping out old parts for new ones. This creates a high demand for precious **metals** which not only increases the price, it increases the amount that must be mined. Mines can be brutal places to work and the manufacturing plants are not much better.

We can help to alleviate some of these issues by recycling and reselling old hardware rather than immediately throwing it out when the new thing comes along.

Service Industries Lose Money

Previously, we mentioned that an increase in the demand for video game developers can lead to an increase in the demand for service industries to support those workers. However, those same service industry jobs can **suffer** as a result of gamers choosing to stay in and play video games instead of venturing out into the world and visiting restaurants and bars and participating in outside activities such as sports.

Although there is nothing wrong with choosing video games over going out to a bar, it is certainly true that service industries receive fewer customers as a result of people **staying at home**. This shouldn’t discourage people from playing video games but it is worth mentioning that economies thrive when money is injected into a variety of industries.

© 2022 Zenva Pty Ltd. All rights reserved.

In this lesson, we will discuss the kinds of data that video games collect and what they potentially do with that data.

What Kinds of Data do Games Collect?

Although not all video games collect data, many of them do in some capacity either through **passive** collection or **actively** having you fill out and submit information. Here is a list of just a few of the types of data that video game companies collect through their video games.

Personal Information

This is information about you, the **user**, such as **name** or **birthday**. Information such as this typically helps populate a profile and can be entered manually or even collected from an existing profile such as an Xbox Live account.

User Demographics Information

This is **population** information such as **location**, **ethnicity**, **age** groups, etc. This populates a population profile rather than an individual profile. Some of this information can be captured passively (location) but much of it is entered manually, if at all (ethnicity, age).

User Habits

This could be a player's habits such as what **time** they usually play, for **how long**, **where** they play, whether they play in bursts or in extended sessions, whether they play **alone** or **with friends,** etc. This could also be habits within the game such as where often people get stuck, which parts of the game people keep coming back to, which parts were **too difficult** or **too easy**, etc. Most of this information is collected and uploaded passively as the game runs. You can sometimes opt out of submitting this kind of information.

Game Feedback

This is typically user feedback about the game such as **reviews** and **ratings**, surveys, questions about what the players do or don't like about the game, etc. This is usually actively collected; games will often ask players to provide some feedback at certain intervals and reviews are usually publicly available and provide a great source of feedback.

How Games Use Data

Although not all video games collect data, the ones that do often use it for **product improvement**. However, some companies use data for more nefarious purposes so it's important to understand how a company will use your data.

Companies must outline exactly what kind of data they collect and what they intend to do with that data, typically in a **Terms and Conditions** sheet. It's always important to read the Terms and Conditions before using any product and to not agree to anything that you don't trust. After all, it's your data and you can choose what to do with it.

Product Improvement

This is one of the best uses of your data. Your feedback (hopefully) goes into improving the product by eliminating or improving areas of a game that players did not like or following themes that they did like. If some parts of a game are reportedly too difficult or too easy then they should be modified to make the **experience** more **enjoyable**. As well, players may give companies ideas for future video games based on what the players found most fun.

© 2022 Zenva Pty Ltd. All rights reserved.

Training for AIs

More and more video games are making use of advanced AIs to make their games more **responsive** and **customized**. This generally increases overall game enjoyment as the game adapts to your play style and can challenge you at an appropriate level. AIs need a lot of data to train on in order to be effective and by passively collecting and analyzing game feedback, these AIs can improve more rapidly to bring you a better gaming experience.

Marketing and Advertising

A lot of companies use your data for marketing and advertising products that you might like based on your search history. For example, if you enjoy playing a lot of first-person shooter games, you're more likely to get **advertisements** for first-person shooter games. If you play only PlayStation then you're likely to get ads for PlayStation games and equipment. This is actually beneficial to us as marketing algorithms are fairly sophisticated and can show us more **products** that we might be **interested in** rather than random items we might never use.

Sell to Third-Party Companies

Unfortunately, a lot of the data that companies collect is sold to other companies for their own gain. Much of this is sold to advertising companies but some of it can be sold to other software companies to help improve their products. For example, a company that specializes in building AIs will likely want to get as much data as they can. This goes into hopefully improving various products and services.

© 2022 Zenva Pty Ltd. All rights reserved.

That concludes our course; congratulations on making it to the end!

In this course, we learned about **object-oriented** techniques and how to build and use objects and classes. We applied this knowledge to build an **escape room game** from scratch. We then discussed how the video game industry affects individuals and broader society. We finished by talking about what data video games collect and how they might use it.

We hope that you enjoyed it and could learn a lot from it and as always, we hope to see you in future courses!

© 2022 Zenva Pty Ltd. All rights reserved.